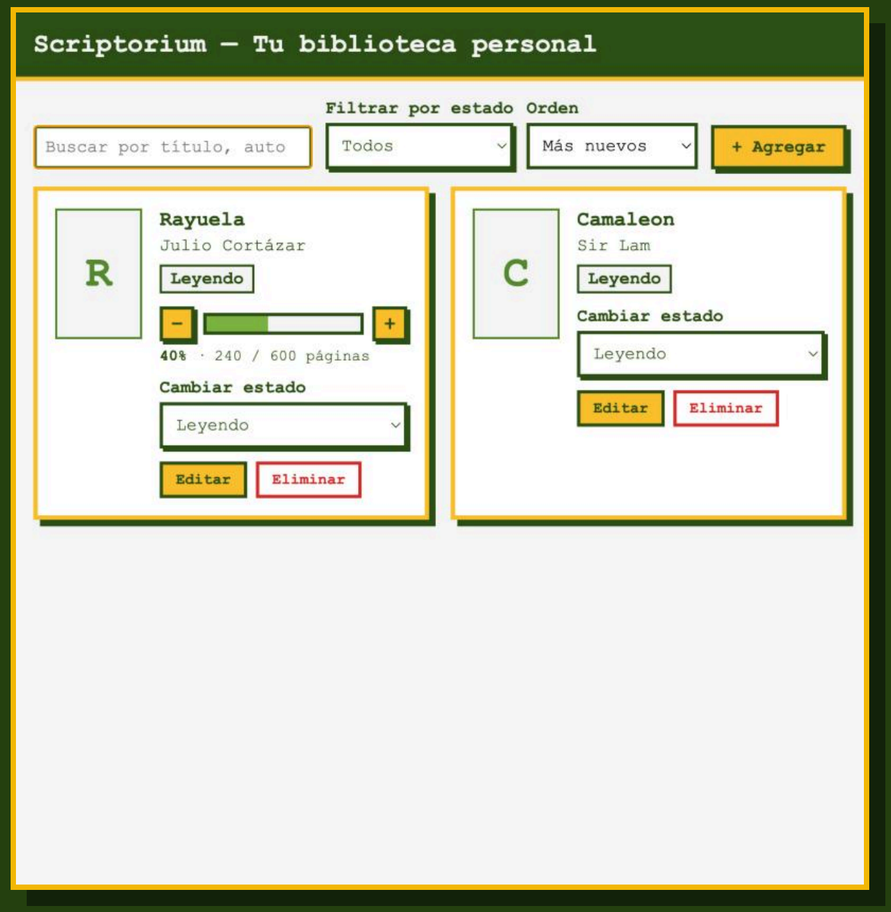
TRABAJO PRÁCTICO FINAL

Scriptorium

Cómo trabajamos con un agente de IA: qué le pedimos, qué planeamos, qué hizo bien y qué hizo mal.



PUNTO DE PARTIDA

Un proyecto anterior como guía, no como código

Tema	Scriptorium original	Esta versión
Datos	localStorage en el navegador	API propia con SQLite
Progreso	readProgress de 0 a 100	pagesRead, validado contra pages
Búsqueda	Google + OpenLibrary vía Firebase	Solo Google, detrás de la API
Cómo se hizo	65 commits previos	Desde cero: spec y plan antes del código

El original sirvió para saber **qué** tenía que hacer la app. El **cómo** salió del circuito de la slide siguiente: primero el plan, después el resto.

EL HARNESS

Primero el plan, después el código

Brainstorming
preguntas de a una

→

Spec y plan
en disco, commiteados

→

Implementa
subagente con TDD

→

Revisa
otro subagente

→

Merge
PR o local

Claude Code

Opus 5.5, seis sesiones

Plugin superpowers

Las skills de brainstorming, writing-plans y subagent-driven-development.

Memoria en disco

CLAUDE.md, specs y planes, y un hook Stop que corre tests y build.

LO QUE SE LE PIDIÓ

Análisis de los 42 prompts

9

Dirección

Fijan el objetivo y las reglas: iniciar desde cero partiendo de un plan.

25

Aprobaciones y elecciones

Respuestas cortas a preguntas del agente: «si», «2», «A», «directo», «procede».

8

Consultas de estado

«¿en dónde te quedaste?», «¿qué falta?», «¿subiste los cambios?».

Funcionó: prompts cortos alcanzan cuando el contexto está en disco y el agente pregunta con opciones. Las 12 decisiones de diseño salieron así.

El riesgo: una aprobación genérica habilita más de lo revisado. Con «procede», el agente escribió todo el spec de portadas de una vez.

EL PLAN

El plan también decía cómo comprobar el trabajo

1. Lo que pedía el plan	2. Lo que hizo el agente	3. Lo que comprobé hoy
Si buscás «100%», que no traiga todos los libros.	Agregó un test: buscar «%» trae solo el libro «100% real».	**Buscar «100%» no trae ningún libro.**
Si llegan datos rotos, que la API responda con un error claro.	Agregó un test que manda un JSON cortado y espera un error 400.	**Responde «Malformed JSON body» y no se cae.**

Los tres planes tenían 15 casos así, cinco por ciclo.

EL PLAN EN LA PRÁCTICA

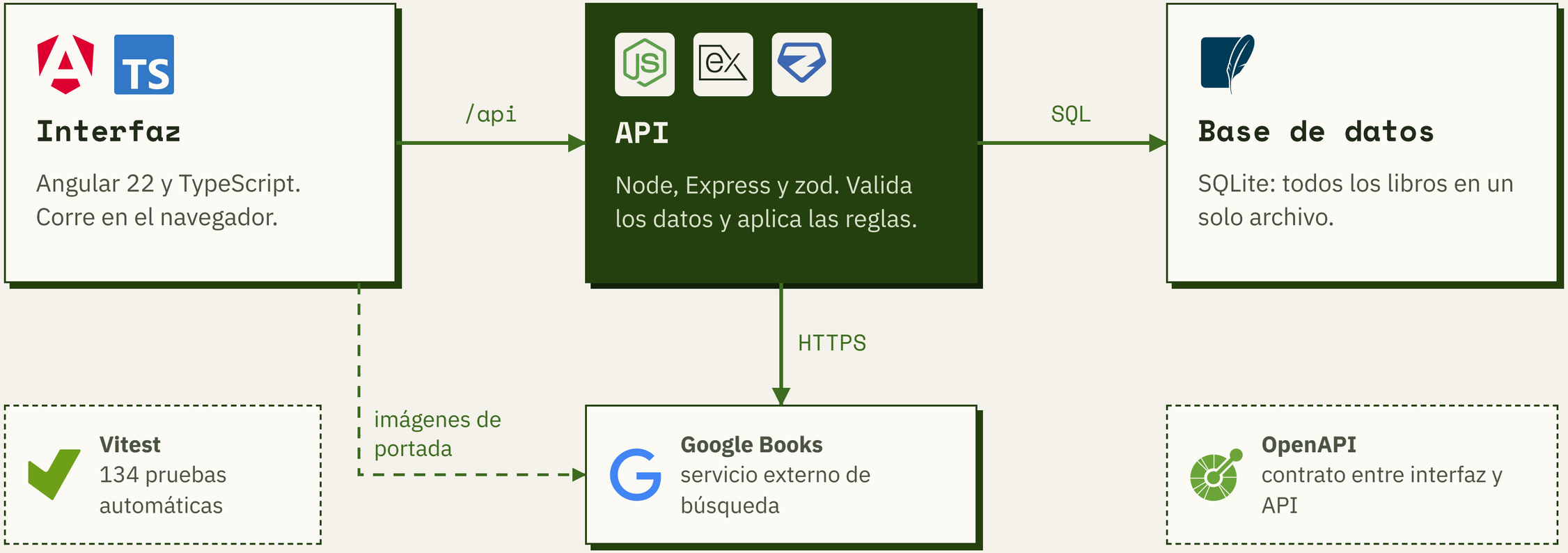
Tres ciclos, cada vez con menos control

Ciclo	Tareas	Implementación	Revisión	Entrada a main
Núcleo (CRUD, estados, progreso)	8	Subagentes con TDD	Por tarea + rama completa	PR #1
Búsqueda en Google Books	5	Subagentes con TDD	Por tarea	Merge local
Portadas	5	Directo, en la sesión principal	Una sola, de la rama	Merge local

El primer ciclo tuvo todos los controles. En el último se saltó parte del brainstorming y se eligió «directo» para ir más rápido, y fue el ciclo donde más cosas quedaron afuera del spec.

EL RESULTADO

Cómo está armado el sistema

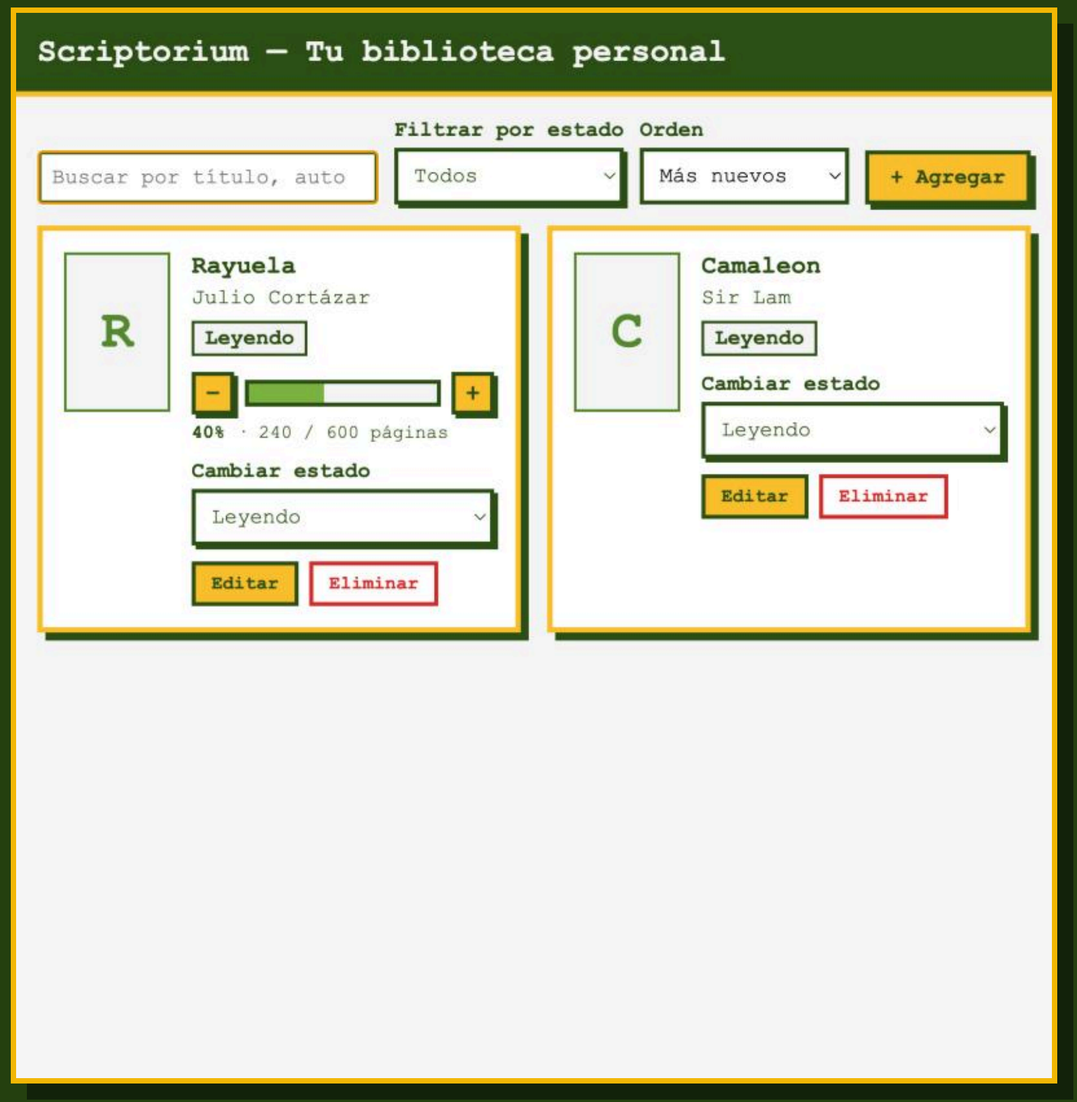


EL RESULTADO

Demo en vivo

Abrir la app →

127.0.0.1:4200 · la API y el front tienen que estar corriendo.



BALANCE

Lo que pedimos y lo que se generó

235
palabras escritas por nosotros, en 42 mensajes

→

15.000
palabras de diseño, planes y documentación

→

3.200
líneas de código, casi la mitad son pruebas

Lo que dice el número

Por cada palabra pedida se generaron unas 14 líneas de código y 64 palabras de documentación.

Lo que no dice

Todo eso hubo que leerlo y aprobarlo. El trabajo humano pasó de escribir código a decidir y revisar.

¿Vale la pena? Sí, si se mantiene la revisión: donde la salteamos, los errores llegaron a la app.

BALANCE

Lo que la IA hizo bien

Preguntó antes de decidir

Hizo una pregunta por vez, con opciones para elegir. Y avisó que no iba a inventar pasos que no se hubieran hecho.

Cumplió lo planeado

Lo que el plan pedía comprobar quedó con pruebas automáticas. Hoy, ante datos con errores, la app avisó en vez de romperse.

Avisó antes de lo delicado

Pidió confirmación antes de publicar algo en internet, y sugirió más cuidado cuando un cambio tocaba los datos reales.

Se revisó a sí misma

Un segundo agente revisó todo junto y encontró errores que se habían escapado, como un formulario con datos del libro anterior.

BALANCE

Lo que la IA hizo mal

Errores del agente

- No fijó las versiones de las herramientas: una instalación futura podía traer otras y romper el proyecto.
- Aplicó dos reglas en el orden equivocado. Cada una sola andaba; juntas fallaban.
- Las instrucciones de instalación pedían una versión vieja de Node, con la que la app no arranca.
- En las portadas no previó qué mostrar si una imagen no carga, ni cómo se veían en el celular.

Decisiones nuestras que costaron

- Aprobamos el diseño de las portadas sin leerlo entero.
- Pedimos ir «directo», sin el agente revisor, para ganar tiempo.
- Dos de las tres partes se sumaron sin dejar registro de revisión.

BALANCE

Qué se puede mejorar

En el trabajo con la IA

- No saltear las preguntas iniciales ni la lectura del diseño escrito.
- Pedir en el plan que pruebe las reglas combinadas, no solo de a una.
- Hacer la revisión completa en cada etapa, no solo en la primera.
- Dejar registro de revisión siempre, aunque el cambio sea chico.

En la app (prueba de hoy)

- Buscar «cortazar» sin tilde no encuentra «Cortázar».
- El progreso se carga de a una página: 600 páginas, 600 clics.
- Un mensaje de error aparece en inglés.
- La búsqueda en Google Books no respondió: falta ver por qué.

PRÓXIMO PASO

Qué falta para usarla de verdad

Acceso

- Usuarios y contraseñas: hoy no hay login.
- Que cada persona vea solo sus libros.
- Conexión segura (https).

Dónde vive

- Un servidor y una dirección web: hoy corre en una sola computadora.
- Copias de seguridad automáticas de los datos.
- Publicar con las pruebas corriendo solas.

Cuando algo falla

- Registro de errores: el fallo de Google Books de hoy no dejó rastro.
- Alertas si la app se cae.
- Arreglar lo pendiente de la slide anterior.

Hoy es una app personal que funciona bien en una computadora. Para producción falta todo lo que rodea al código.